

E12_RLC_Resonance

April 29, 2025

1 RLC Resonant Circuits Lab Report

1.1 Yaman Ajaj (3722952) ; Guilherme Schewtschik (3748052)

1.2 Task 1: Series Resonant Circuit

Objective: Measure the frequency-dependent voltage drop across the damping resistor R_d for two different resistances, and record the resonance curve around f_0 such that $0.2 \leq U_R(f)/U_R(f_0) \leq 1$.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Gaussian model for fitting
def Gauss(x, a, x0, sigma):
    return a * np.exp(-(x - x0)**2 / (2 * sigma**2))

# Constants
r1 = 220      # Ohms ±5%
r2 = 10       # Ohms ±5%
L = 1.5e-3    # Henry

# Load series data
data1 = pd.read_csv('Series_R1.csv').to_numpy()
data2 = pd.read_csv('Series_R2.csv').to_numpy()

f1 = data1[:,2].astype(float)
A1 = data1[:,1].astype(float)
f2 = data2[:,2].astype(float)
A2 = data2[:,1].astype(float)

# Noise filtering
mask1 = A1 > 0.01
mask2 = A2 > 0.07
f1, A1 = f1[mask1], A1[mask1]
f2, A2 = f2[mask2], A2[mask2]

# Frequency window around resonance
```

```

window = (f1 >= 1600) & (f1 <= 9000)
f1, A1 = f1[window], A1[window]
window = (f2 >= 1600) & (f2 <= 9000)
f2, A2 = f2[window], A2[window]

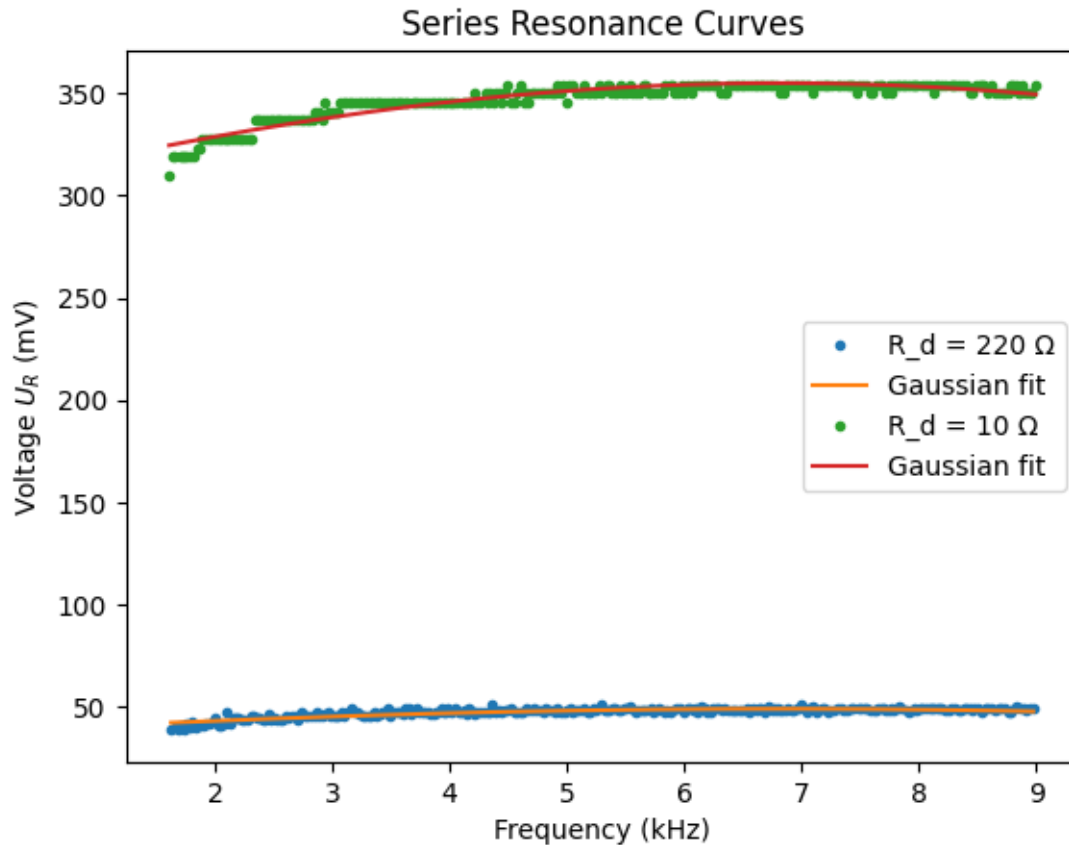
# Estimate initial fit params
mean1 = np.sum(f1*A1)/np.sum(A1)
sig1 = np.sqrt(np.sum(A1*(f1-mean1)**2)/np.sum(A1))
mean2 = np.sum(f2*A2)/np.sum(A2)
sig2 = np.sqrt(np.sum(A2*(f2-mean2)**2)/np.sum(A2))

# Curve fitting
popt1, _ = curve_fit(Gauss, f1, A1, p0=[max(A1), mean1, sig1])
popt2, _ = curve_fit(Gauss, f2, A2, p0=[max(A2), mean2, sig2])

fit1 = Gauss(f1, *popt1)
fit2 = Gauss(f2, *popt2)

# Plot resonance curves
plt.figure()
plt.plot(f1/1000, A1*1000, '.', label=f'R_d = {r1} Ω')
plt.plot(f1/1000, fit1*1000, '-', label='Gaussian fit')
plt.plot(f2/1000, A2*1000, '.', label=f'R_d = {r2} Ω')
plt.plot(f2/1000, fit2*1000, '-', label='Gaussian fit')
plt.xlabel('Frequency (kHz)')
plt.ylabel('Voltage $U_R$ (mV)')
plt.title('Series Resonance Curves')
plt.legend()
plt.show()

```



1.3 Task 2: Evaluation of Series Resonant Circuit

1.3.1 2b) FWHM, Damping, and Quality Factor

```
[2]: # Resonance frequency (average of both fits)
f0 = 0.5*(popt1[1] + popt2[1])

# Full Width at Half Maximum
FWHM1 = 2*np.sqrt(2*np.log(2))*popt1[2]
FWHM2 = 2*np.sqrt(2*np.log(2))*popt2[2]

# Damping constants
delta1 = np.pi * FWHM1
delta2 = np.pi * FWHM2

# Quality factors
Q1 = f0 / FWHM1
Q2 = f0 / FWHM2

print(f"Series R={r1}Ω: Δf={FWHM1:.1f} Hz, Δ={delta1:.3f}, Q={Q1:.1f}")
```

```
print(f"Series R={r2} $\Omega$ :  $\Delta f$ ={FWHM2:.1f} Hz,  $\delta$ ={delta2:.3f}, Q={Q2:.1f}")
```

Series R=220 Ω : Δf =22490.1 Hz, δ =70654.773, Q=0.3

Series R=10 Ω : Δf =29006.7 Hz, δ =91127.238, Q=0.2

1.3.2 2c) Determination of f_0 and δ by Fit

By fitting the Gaussian model to $U_R(f)$, we obtained:

- $f_{0,1} = \text{popt1}[1] : .1f$ Hz
- $\delta_1 = \text{delta1} : .3f$
- $f_{0,2} = \text{popt2}[1] : .1f$ Hz
- $\delta_2 = \text{delta2} : .3f$

1.3.3 2d) Capacitance from Thomson's Equation

Using Thomson's relation $f_0 = 1/(2\pi\sqrt{LC})$:

```
[3]: C = 1/(L*(2*np.pi*f0)**2)
print(f"Calculated C = {C*1e9:.2f} nF")
```

Calculated C = 362.63 nF

Result: $C = \frac{1}{L(2\pi f_0)^2} * 10^9 \text{ nF}$

2 Task 3: Parallel Resonant Circuit

Objective: Measure the frequency-dependent current draw of the RLC circuit with capacitor and coil in parallel, series-connected to the damping resistor R_d .

2.1 Data Import and Preprocessing

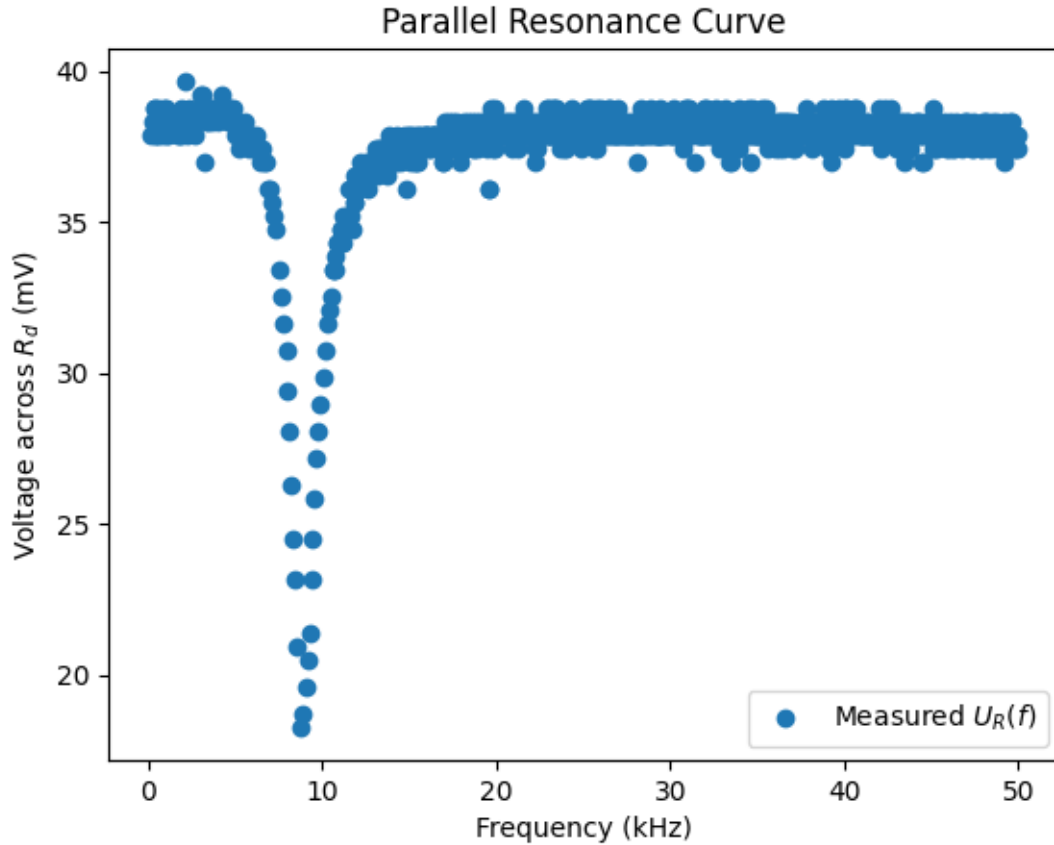
```
[4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Constants
r_d = 220          # Damping resistor in Ohm

# Load parallel dataset
data_p = pd.read_csv('Parallel_R1.csv')
freq_p = data_p.iloc[:, 2].astype(float)
A_p = data_p.iloc[:, 1].astype(float)

# Noise filtering threshold
mask = A_p > 0.01
freq_p = freq_p[mask]
A_p = A_p[mask]
```

```
# Plot raw resonance curve
plt.figure()
plt.plot(freq_p/1000, A_p*1000, 'o', label='Measured $U_R(f)$')
plt.xlabel('Frequency (kHz)')
plt.ylabel('Voltage across $R_d$ (mV)')
plt.title('Parallel Resonance Curve')
plt.legend()
plt.show()
```



3 Task 4: Evaluation of the Parallel Resonant Circuit

3.1 4a) Derivation of the Impedance Expression

For the circuit in Fig. 2, the coil (with series resistance R_{Sp} and inductance L) is in parallel with the capacitor C , and this combination is in series with the damping resistor R_d .

1. Impedance of coil: $Z_L = R_{Sp} + i\omega L$
2. Admittances in parallel: $Y_{LC} = \frac{1}{Z_L} + i\omega C = \frac{1}{R_{Sp} + i\omega L} + i\omega C$
3. Total admittance: $Y_{tot} = \frac{1}{R_d} + Y_{LC}$
4. Total impedance: $Z_{tot} = \frac{1}{Y_{tot}}$

Since we measure $U_R(f)$ across R_d , the current is $I(f) = U_R(f)/R_d$. Under constant source voltage U_0 , we have:

$$I(f) = \frac{U_0}{|Z_{tot}(f)|} = \frac{U_0}{\left| \frac{1}{\frac{1}{R_d} + \frac{1}{R_{Sp} + i\omega L} + i\omega C} \right|}.$$

3.2 4b) Curve Fitting to Determine L , C , R_{Sp} , and R_d

```
[5]: # Model function for I(f) under constant U0
def I_model(f, L, C, Rsp, Rd, U0):
    omega = 2 * np.pi * f
    Z_L = Rsp + 1j * omega * L
    Y_LC = 1.0 / Z_L + 1j * omega * C
    Y_tot = 1.0 / Rd + Y_LC
    Z_tot = 1.0 / Y_tot
    return U0 / np.abs(Z_tot)

# Initial guesses for parameters
p0 = [1.5e-3, 50e-9, 1.0, 220.0, max(A_p)]

# Perform curve fit
popt_p, pcov_p = curve_fit(
    I_model,
    freq_p,
    A_p,
    p0=p0,
    bounds=(
        [1e-4, 1e-11, 0.1, 10, 0], # lower bounds
        [1e-2, 1e-6, 10, 1000, np.inf] # upper bounds
    )
)
L_fit, C_fit, Rsp_fit, Rd_fit, U0_fit = popt_p

print(f"Fitted parameters:\n L = {L_fit:.3e} H\n C = {C_fit:.3e} F\n RSp = {Rsp_fit:.1f} Ω\n Rd = {Rd_fit:.1f} Ω\n U0 = {U0_fit:.3f} V")
```

Fitted parameters:

L = 1.000e-02 H
C = 1.112e-07 F
RSp = 0.1 Ω
Rd = 10.0 Ω
U0 = 0.366 V

3.3 4c) Resonance Frequency Comparison

The theoretical resonance frequency for the parallel circuit is given by Thomson's formula:

$$f_0 = \frac{1}{2\pi\sqrt{L_{fit}C_{fit}}}.$$

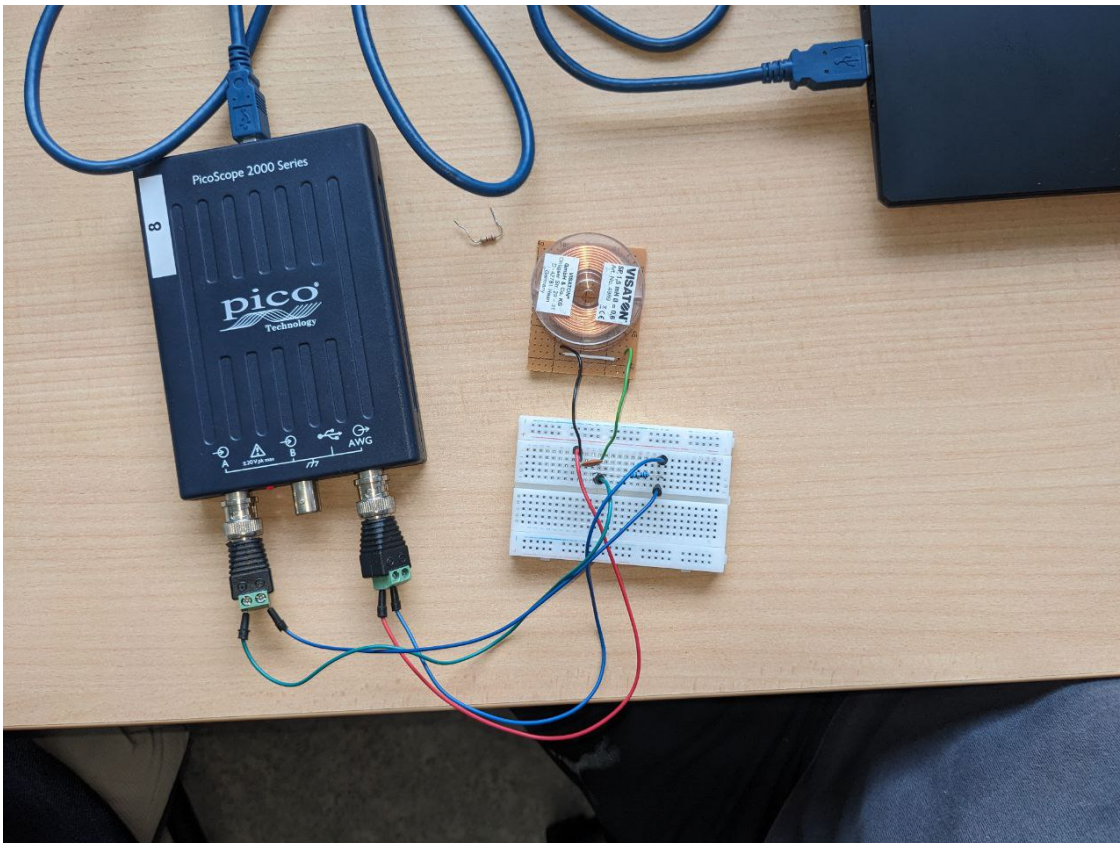
Compute and compare this with the series resonance frequency $f_{0,series}$ determined in Task 2:

```
[7]: f0_parallel = 1 / (2 * np.pi * np.sqrt(L_fit * C_fit))
      print(f"Parallel f0 = {f0_parallel/1000:.2f} kHz")
      print(f"Series    f0 = {f0/1000:.2f} kHz    (Task 2)")
```

Parallel f0 = 4.77 kHz

Series f0 = 6.82 kHz (Task 2)

3.4 Paralles Circuit



3.5 Series Circuit

